Chaos Engineering

break it on purpose, at a time you choose

Watch the 4-min video

In one line: chaos engineering runs controlled, hypothesis-driven experiments that inject real failures (killed pods, latency, errors, lost zones) and checks whether steady-state metrics hold, starting tiny with automatic abort conditions, so you find weaknesses before your customers do.

1 Have you ever seen your failover work?

The system has failover (lesson 11), retries (lesson 5) and circuit breakers (lesson 6). But nobody has watched them work. Many teams discover during a real outage that failover was misconfigured or a timeout was ten times too long. Chaos engineering finds those gaps **on purpose**, during working hours, with the team watching.

2 An experiment, not random breakage

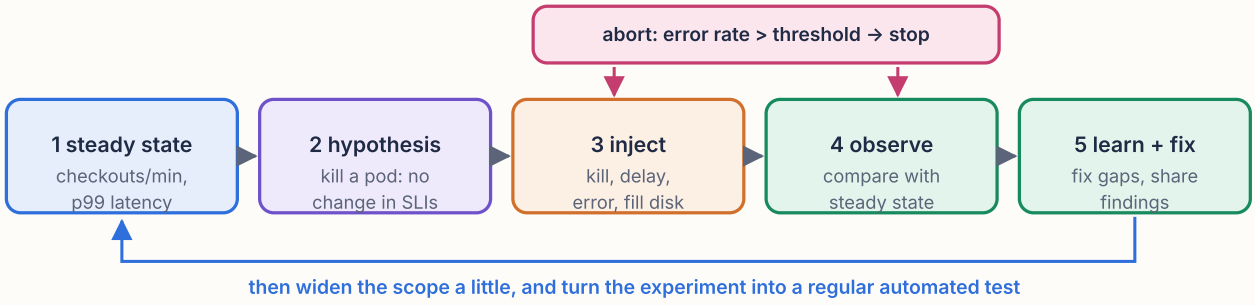


Figure 1. The chaos experiment loop. The abort condition is defined before anything is injected.

Define the **steady state** as user-facing metrics, e.g. successful checkouts per minute or p99 latency (your SLIs, lesson 2). State a **hypothesis**: "if we kill one instance, the steady state won't change". **Inject** a realistic failure and compare. If the metrics hold, confidence grows; if not, you've found a weakness cheaply.

Fault	Example injection	What it tests
Instance / pod loss	terminate a random pod or VM	redundancy, health checks, session handling
Network	add 300 ms latency or 5% packet loss to a dependency	timeouts, retries, circuit breakers
Dependency errors	make a downstream return 503s	fallbacks and graceful degradation
Resources	fill a disk, burn CPU, exhaust memory	alerts, autoscaling, limits
Zone / region	block a whole zone's traffic	failover, capacity of the remaining zones

3 Safety first: start small, widen slowly

Start with **one instance, in staging, in business hours**, with the team watching. Define **abort conditions** in advance, such as error rate above 1–2%, and stop automatically when they are crossed. Widen the scope only after each step passes.

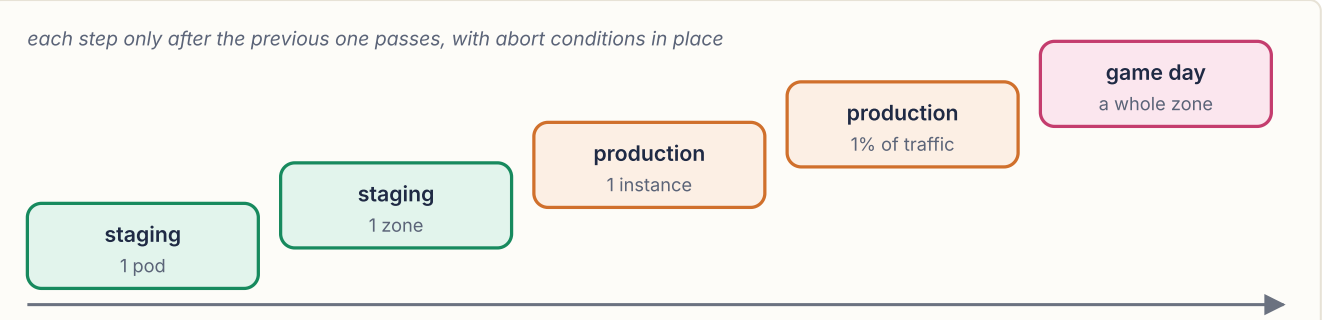


Figure 2. Growing the blast radius step by step.

A **game day** is a planned session where the team runs bigger experiments together, such as failing over a database or losing a zone, and practises the incident response (lesson 70). Tools: Netflix's **Chaos Monkey** randomly terminates instances; **LitmusChaos** and **Chaos Mesh** run experiments in Kubernetes; **AWS Fault Injection Service** injects faults into cloud resources with stop conditions tied to CloudWatch alarms; Azure has Chaos Studio.

Typical findings: a timeout that's too long, a retry storm, an alert that never fires, a stale runbook, a dependency nobody knew about. Each becomes a fix, results are shared, and the experiment becomes a **regular automated test** so the weakness can't quietly return.

WORKED EXAMPLE: KILLING A POD EVERY TUESDAY

A scheduled job kills **one random pod of each service** in staging every Tuesday. Hypothesis: requests keep succeeding and the pod is replaced within a minute. Abort condition: staging error rate **above 2%**. Week one, checkout failed: it kept user sessions **in memory**, so users on the killed pod were logged out. The team moved sessions to **Redis**; the experiment has passed every week since and now guards against the regression.

COMMON MISTAKES

- Starting in production with a big blast radius and no abort condition.
- No steady-state metric, so nobody can say whether the experiment passed.
- Running experiments at night or without the owning team watching.
- Finding weaknesses and not fixing them, or not repeating the experiment.

4 Key terms

Chaos engineering: controlled experiments that inject failures

Hypothesis: the expected result: steady state holds

Abort condition: threshold that stops the experiment automatically

Steady state: metrics that show normal, healthy behaviour

Blast radius: how much an experiment can affect

Game day: planned team exercise with bigger failures

5 Interview questions

Q1. What is chaos engineering? BASIC

Running careful, hypothesis-driven experiments that inject real failures to check that the system's steady state holds, and fixing what breaks.

Q2. What is a game day? BASIC

A planned session where the team injects bigger failures together, like a database failover or a lost zone, and practises the incident response.

Q3. How would you introduce chaos engineering safely? SCENARIO

Define steady-state metrics and a hypothesis, start with a tiny blast radius in staging during business hours, set automatic abort conditions, widen gradually, and turn findings into fixes and automated experiments.

Q4. Which experiment would test your timeouts and circuit breakers? SCENARIO

Inject latency or errors into one dependency, e.g. 300 ms delay or 503s, and check that callers time out, open the breaker and serve fallbacks while SLIs hold.

Q5. Isn't chaos engineering just breaking things randomly? TRICKY

No: each experiment has a hypothesis, a measured steady state, a limited blast radius and abort conditions; the goal is evidence, not damage.

Q6. Why run experiments in production at all? TRICKY

Staging differs in traffic, data, config and scale; some weaknesses only appear in production, so once staging passes, small controlled production experiments give real confidence.

Next: SD-K-018 · Static Stability backendroadmap.com